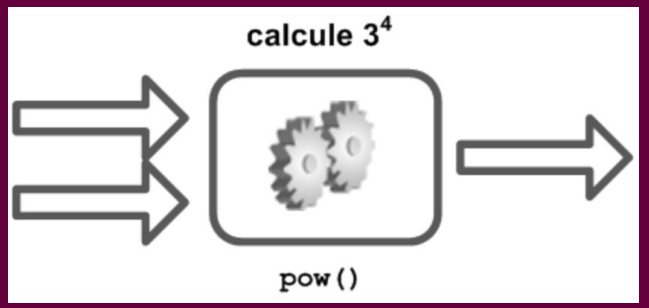
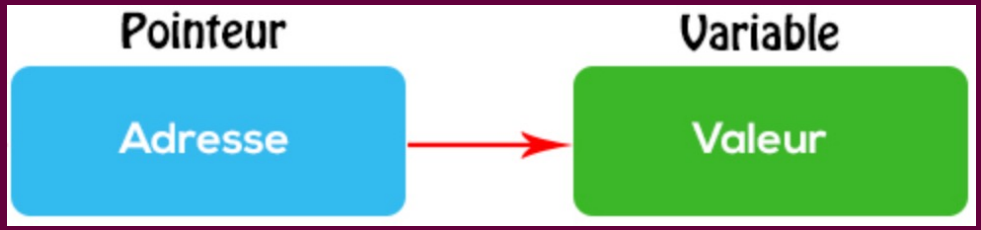
université PARIS-SACLAY | IUT DE CACHAN

Programmation C / C++:
Fonctions / Pointeurs





BUT 3 ESE – LP MECSE
v1.8 – Sept. 2025

Les fonctions - principes

Une fonction :

- **contient** des opérations qui seront ***exécutées*** à chaque appel
- peut **retourner une** valeur à la fonction appelante
- peut **recevoir** des paramètres

Exemple:



On veut calculer 3^4, soit 3 à la puissance 4:

syntaxe de la fonction : **double pow(double x,double y);**

utilisation: result = pow(3,4);

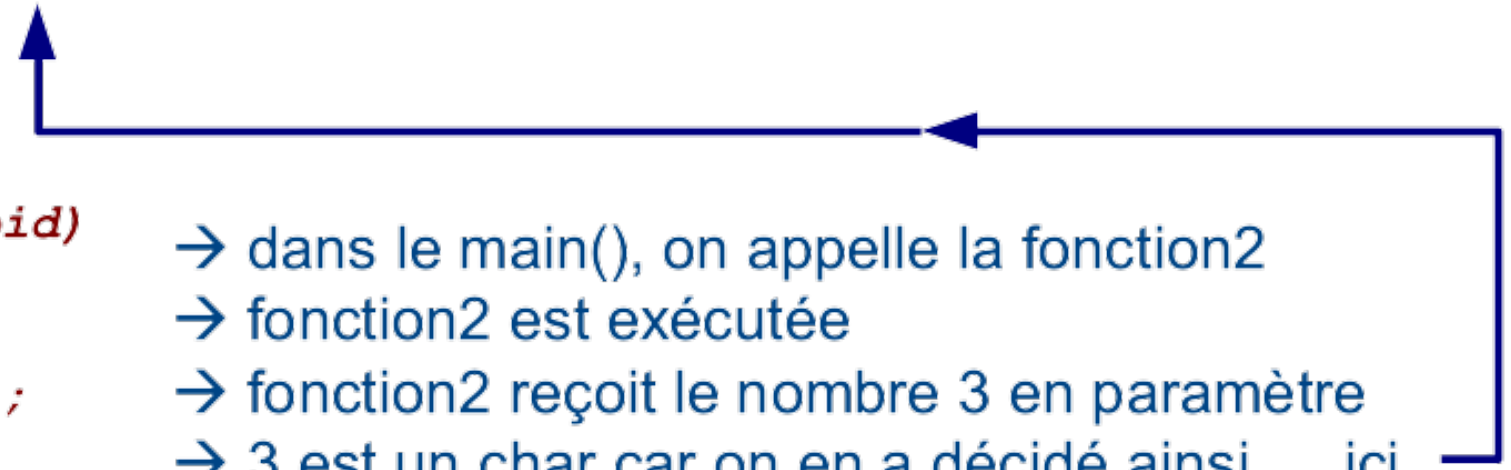
quel doit être le type de result?...

Les fonctions - principes

Exemples (suite) :

```
void fonction2(char x)
{
...
}

void main (void)
{
...
fonction2(3) ;
...
}
```



→ dans le main(), on appelle la fonction2
→ fonction2 est exécutée
→ fonction2 reçoit le nombre 3 en paramètre
→ 3 est un char car on en a décidé ainsi … ici

```
void main (void)
{
char y=5 ;
fonction2(y) ;
...
}
```

→ ici aussi, on appelle la fonction2
→ fonction2 est exécutée
→ fonction2 reçoit le nombre 5 (char) en paramètre

Les fonctions - principes

Exemples (suite) :

```
int fonction3(void)
{
...
return 5 ;
}

void main (void)
{
int z ;
z=fonction3() ;
...
}
```

→ dans le main(), on appelle la fonction3
→ fonction3 est exécutée
→ fonction3 retourne le nombre 5 en paramètre
→ 5 est un int car on en a décidé ainsi … ici

Une fonction retourne 1 paramètre UNIQUE
(1 nombre, 1 caractère …)

Définition d'une fonction

Pour définir une fonction, il faut donc indiquer:

- *Son nom*
- *La liste des paramètres (type + noms)*
- *Le type de la valeur de retour*

Remarque : *On va écrire ces définitions dans un fichier source .c, qu'il faudra rajouter au projet*

Exemple de définition:

```
void affiche_bonjour (void)
{
    printf ("Bonjour tout le monde\n");
}
```

Organisation des fichiers

Exemple : un fonction additionnant 2 entiers



main.c*

```c
#include <stdio.h>
#include "fonction.h"

int main()
{
    int result;

    result=somme(5,6);

    return 0;
}
```

fonction.c*

```c
int somme ( int val1, int val2 )
{
    int resultat;

    resultat = val1 + val2;

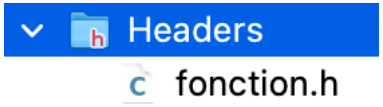
    return resultat ;
}
```

Variables **locales** à la fonction

Il ne faut pas les redéclarer !

Au moment de l'appel, il faut absolument que la fonction soit **DÉCLARÉE.**
Ces déclarations (**PROTOTYPES**) sont écrites dans un fichier d'en-tête .h

➔Pour utiliser une fonction, on doit trouver son **prototype**!



fonction.h*

```c
#ifndef FONCTION_H
#define FONCTION_H

int somme ( int val1, int val2 );

#endif // FONCTION_H
```

Appel de fonction

- Règle :
  A PARTIR DU PROTOTYPE, l'appel se fait en donnant le **nom** de la fonction, suivi de **ses arguments entre parenthèses**

- Important :
  Même si elle ne prend pas d'arguments, il faut utiliser les parenthèses vides **( )**

  - Exemple :

```
int main ( )
{
    int res, nb = 5;

    bonjour ( );
    res = somme ( nb, 3) ;
...
```

Pas d'arguments!

La valeur retournée par *somme* est stockée dans *res*

Variables : global, local, static

- Règles : variables locales
  - Toute variable déclarée **dans le corps** d'une fonction est **locale** à cette fonction
  - Leur portée est **limitée à la fonction**

- Règles : variables globales
  - Toute variable déclarée **en dehors** du corps de toute fonction est dite **globale**
  - Portée : **toutes les fonctions** à l'intérieur du fichier source où elle est déclarée y ont accès, en lecture et écriture

N.B : variables globales = sources d'erreurs

⇨ éviter de les utiliser (sauf cas particuliers)

Variables : global, local, static

- Gestion mémoire :
  - Variables locales : crées lors de l'appel de la fonction, détruites à la sortie de cette fonction
  - Variables globales : crées lors de l'exécution du programme, détruites en quittant
- Cas particulier : variables **statiques**
  - Variable locale à une fonction (portée limitée) mais qui conserve sa valeur lorsque la fonction se termine
  - **Initialisée à *0*** *(ou à la valeur indiquée) au 1er appel*
  - Intérêt : mémorisation (exemple : compteur…)
  - Exemple :

Mot-clé à utiliser →

```
int ma_fnct ( float valeur )
{
    static unsigned int nb_appels;
    ...
```

Passage d'arguments

• Fonction « basique » : elle permute 2 valeurs passées en argument. ***Passage par VALEUR***:

```
void echange ( int x, int y )
{
  int tampon;
  tampon = x;
  x = y;
  y = tampon;
}
```

```
int main( )
{
  int a = 5, b = 8;
  printf("a=%d b=%d\n", a, b);
  echange (a, b);
  printf("a=%d b=%d\n", a, b);
}
```

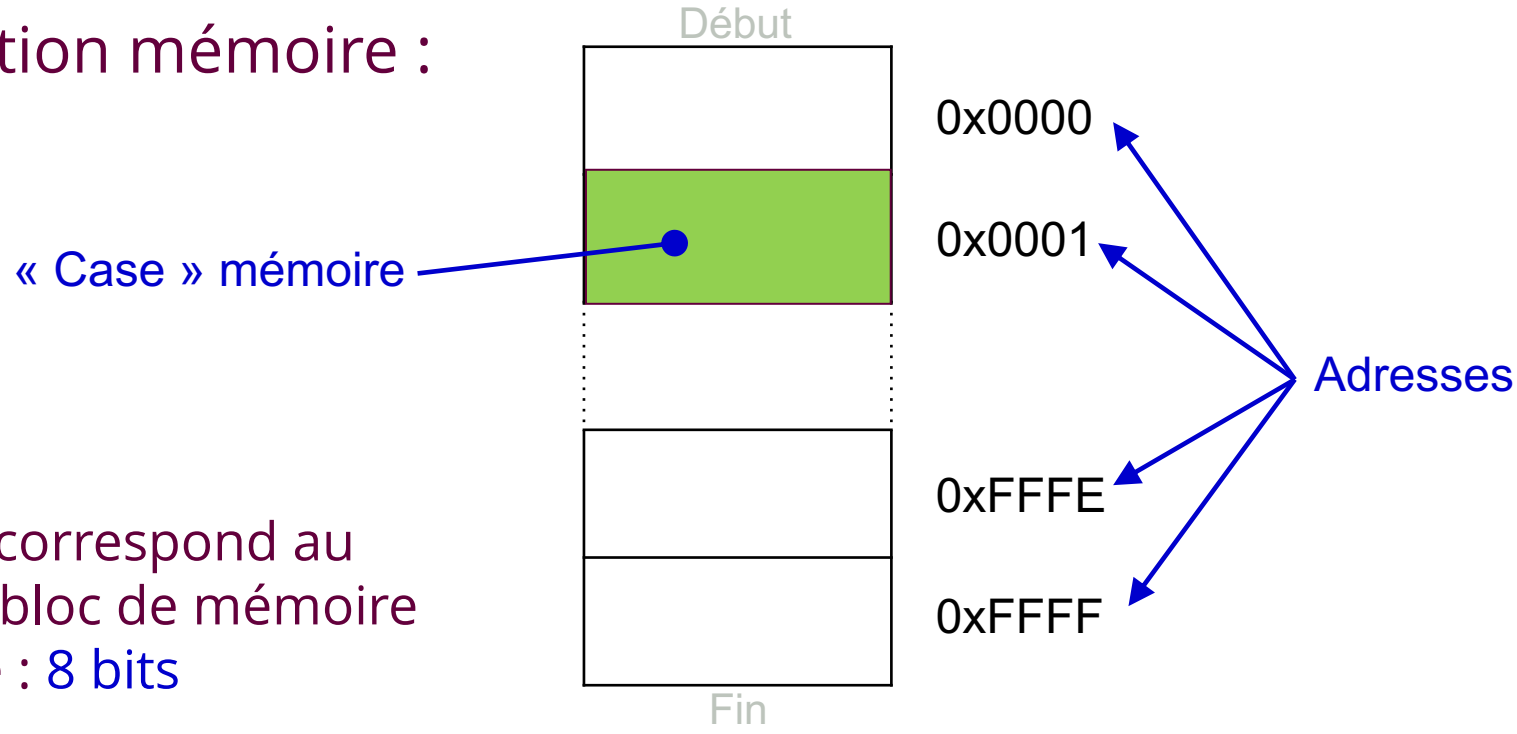
Quel est l'affichage obtenu à l'écran ? Pourquoi ?

APPLICATION: Créer une fonction ***offset***, qui ajoute une quantité entière (offset passé en paramètre) à 1 valeur passée elle aussi en paramètre. Elle retourne le résultat. **Faire valider**

⇨ Il faut passer les arguments **par adresse (cf page 22)**

Espace mémoire & adressage

Organisation mémoire :



Une case correspond au plus petit bloc de mémoire accessible : 8 bits

Chaque adresse est unique.
Par extension, on parle de l'adresse d'une variable (ou fonction) stockée **à partir** d'une case mémoire donnée.

Notion de pointeur

- Un pointeur :
  - = une variable particulière : contient une **adresse mémoire**
  - « pointe » sur le début d'un ensemble de cases
  - possède un type : celui du contenu de l'objet pointé
- Déclaration :

```
type   *  identificateur;
```

Type de l'objet pointé

!!

Distinction déclaration pointeur / variable

Nom du pointeur

- Exemple :

```
int *pi;
float *pr;                          // pi, pr, pc1, pc2 et pcar sont des pointeurs
char *pc1, *pc2, ab, *pcar;
```

Déréférencement et indirection

- Opérateur de déréférencement : *****
  - Permet de récupérer le **contenu** de la case mémoire pointée
  - Ne s'applique qu'à un pointeur
- Opérateur d'indirection : **&**
  - Permet de récupérer **l'adresse** à partir de laquelle une variable est stockée
  - Ne s'applique qu'à une variable

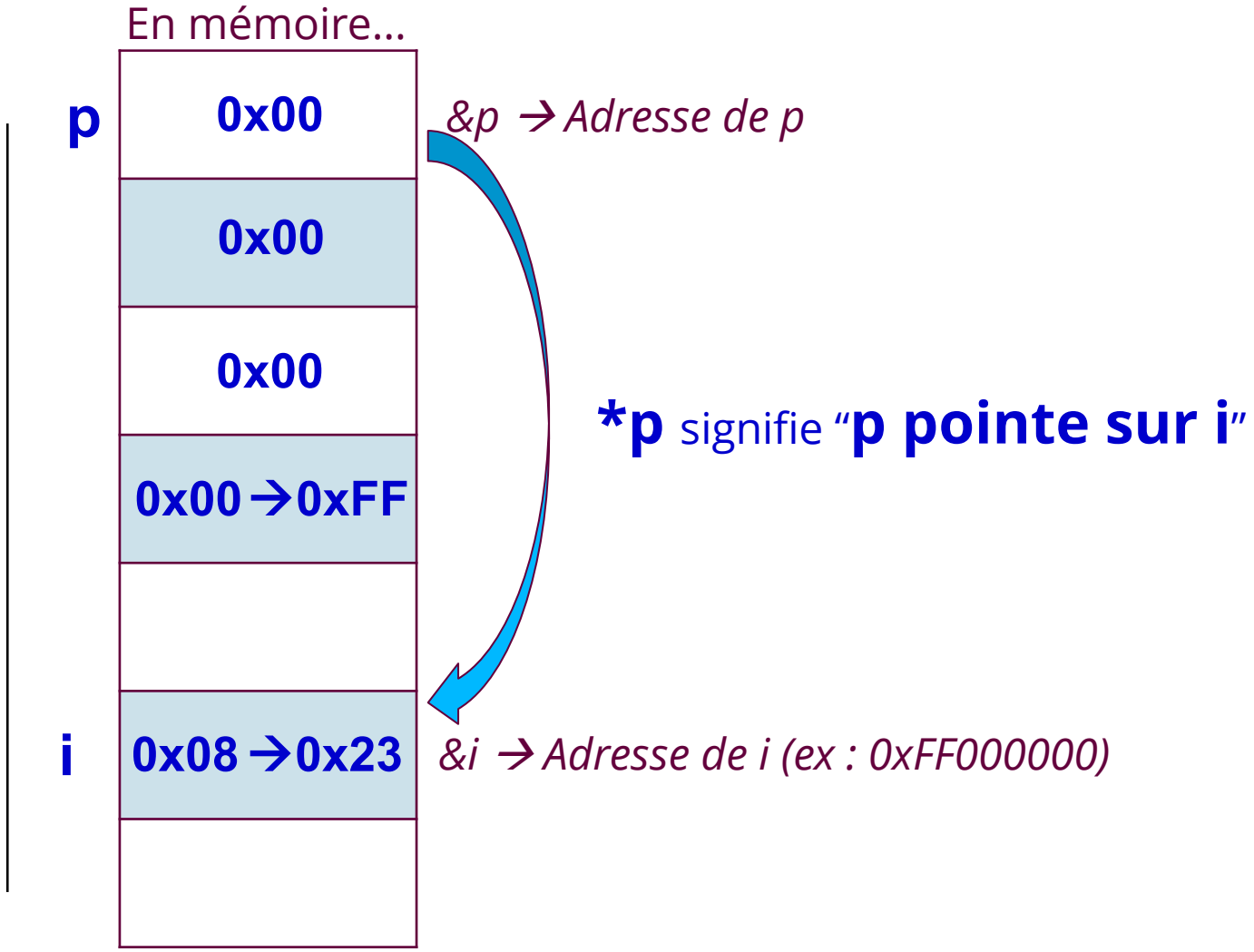
N.B : pour éviter les erreurs de segmentation, toujours **initialiser** un pointeur qui vient d'être déclaré, au moins avec la valeur **NULL**

Déréférencement et indirection

- Exemple :

Code source

```
int i=8;
int *p=NULL;
p=&i;
*p=0x23;
```



Arithmétique des pointeurs

- Opérations arithmétiques sur les pointeurs :
  - Incrémentation
  - Décrémentation
  - Soustraction de deux pointeurs
- Objectifs :
  - Déplacer un pointeur dans le plan mémoire
  - Accéder séquentiellement à des données contigües en mémoire
- Principe : *int val, *p=&val, i=3;*

⇨ p + i = (adresse contenue dans p + (i x taille(élément pointé par p)))

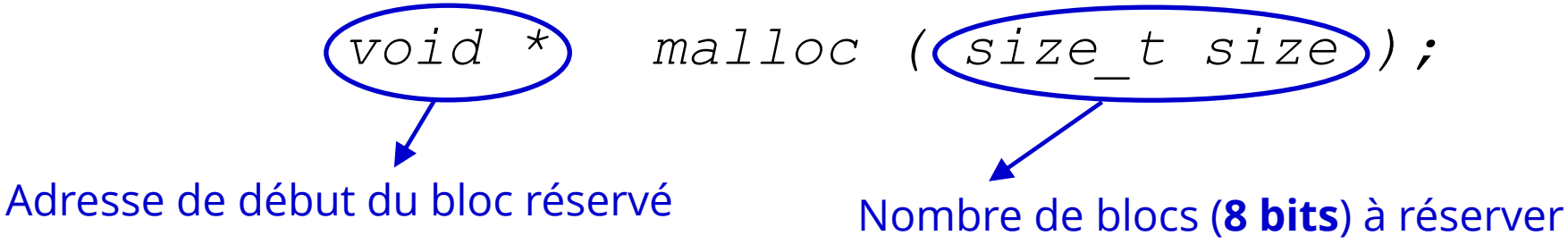
Allocation dynamique de mémoire

- Intérêt :
  - Réservation, **en cours d'exécution**, de blocs mémoires de taille quelconque
  - Complète l'allocation statique (avant compilation)
  - Obligatoire dans de nombreux cas (cf. gestion d'images)
- Inconvénients :
  - Manipulation assez fastidieuse, surtout en C
  - Syntaxe « longue »
  - Aucune vérification des manipulations de données $\Rightarrow$ risques d'erreurs de segmentation !

Adapté uniquement quand un **système d'exploitation** gère la mémoire disponible d'un système **multitâches** !

Allocation dynamique de mémoire

- Opérateurs et fonctions utiles :
  - *sizeof (type ou variable)*
  - *void * malloc (size_t size);*
  - *void free (void * ptr);*
- Utilisation :



➪ Retourne NULL si allocation impossible

➪ Transtypage obligatoire en fonction du type de données à allouer

Allocation dynamique en C++

- Nouveaux opérateurs spécifiques :

Allocation d'une variable unique

- *new type*
- *delete pointeur*

Allocation d'un tableau dynamique

- *new type[taille]*
- *delete[ ] pointeur*

Les crochets sont **indispensables**

⇨ *new* retourne NULL si l'allocation est impossible (et devrait générer une exception selon la norme ANSI)

⇨ Aucun transtypage nécessaire ⇒ syntaxe moins lourde et plus intuitive qu'en C

⇨ Il ne faut **jamais** utiliser à la fois les opérateurs C++ et ceux du C pour gérer un espace mémoire donné (ex. : malloc puis delete)

Allocation dynamique en C++

- Utilisation :

Variable unique

```
int *pentier = NULL;

pentier = new int; // allocation dyn.

*pentier = 10;
cout << "Valeur=" << *pentier << endl;
...
delete pentier;
```

Tableau dynamique

```
int *ptab = NULL, nb_val = 1000, i;

ptab = new int[nb_val]; // allocation
                        // dynamique

for(i=0; i<nb_val ; i=i+1)
{
  *(ptab+i) = 0; // initialisation a 0 des
                 // cases du tableau
}
...
delete[] ptab;
```

Comparaison C/C++

Exemple (C vs C++) : allocation d'un tableau de *nb* entiers positifs, initialisation et affichage du contenu

```c
unsigned int i, nb;
int *ptab = NULL;
printf("Nombre de valeurs ?\n"); scanf("%d", &nb);

ptab = (int*)malloc(sizeof(int)*nb);
if (ptab != NULL)
{
   for (i=0 ; i<nb ; i++)
      *(ptab+i) = i;
   for (i=0 ; i<nb ; i++)
      printf("%d\t", *(ptab+i));
   printf("\n");
   free(ptab);
}
else
   printf("Allocation impossible\n");
```

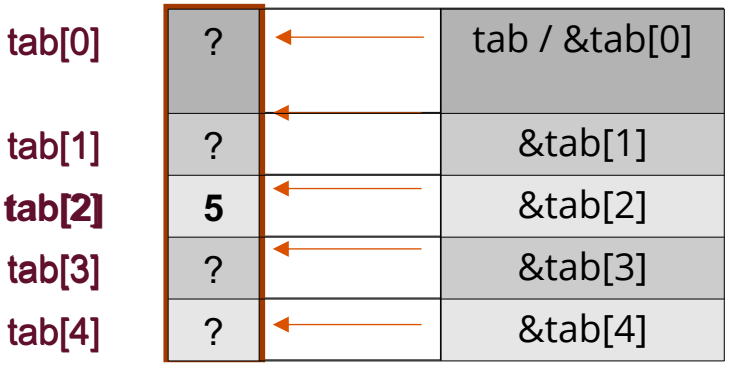
```c
unsigned int i, nb;
unsigned int *ptab = NULL;
cout<<"Nombre de valeurs ?<<endl; cin>>nb;

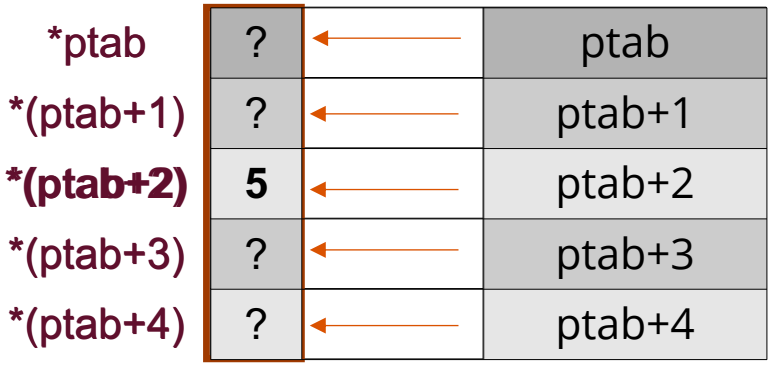
ptab = new unsigned int[nb];
if (ptab != NULL)
{
   for (i=0 ; i<nb ; i++)
      *(ptab+i) = i;
   for (i=0 ; i<nb ; i++)
      cout << *(ptab+i) << "\t";
   cout << endl;
   delete[] ptab;
}
else
   cout << "Allocation impossible"  << endl;
```

Correspondance Tableau / Pointeurs

NOTATION TABLEAU **NOTATION POINTEUR**

```
short tab[5];          short * ptab;
                       ptab = malloc(5*2);  // 5*2octets
tab[2] = 5;            *(ptab+2) = 5 ;
```





Passage d'argument par adresse

- Exemple : *void modif_tab (double *mon_tab);*

  Paramètre = pointeur ⇨ Passage d'argument par adresse

- Intérêt :
  - Avec return, impossible de retourner plusieurs valeurs
    ⇒ on donne l'adresse des valeurs à modifier **grâce aux arguments**
  - Fonctions qui travaillent sur des **tableaux** : passage par adresse obligatoire (et implicite)

- illustration concrète :
  - Objectif : créer une fonction qui échange le contenu de deux variables passées en arguments

Passage d'argument par adresse

- Fonction correcte : passage par adresses

```
void echange ( int *x, int *y )
{
  int tampon;
  tampon = *x;
 *x = *y;
 *y = tampon;
}
```

```
int main( )
{
  int a = 5, b = 8;
  printf("a=%d b=%d\n", a, b);
  echange ( &a, &b);
  printf("a=%d b=%d\n", a, b);
}
```

Résultat : a et b ont bien été échangées après l'appel de la fonction ***echange***

Passage d'argument par référence
(uniquement en C++)

- Fonction correcte : passage par références

Syntaxe : déclaration d'une référence sur un entier

```
void echange ( int &x, int &y )
{
  int tampon;
  tampon = x;
x = y;
y = tampon;
}
```

Rien !

```
int main( )
{
  int a = 5, b = 8;
  printf("a=%d b=%d\n", a, b);
  echange (a, b);
  printf("a=%d b=%d\n", a, b);
}
```

Rien !

Résultat : a et b ont bien été échangées après l'appel de la fonction *echange*



Différence (syntaxe) très légère / passage par valeurs !

Surcharge de fonction
(Uniquement en C++)

- Nouveauté :
  - En C++, il est possible de déclarer plusieurs fonctions portant le **même nom**, mais dont les paramètres sont en nombre ou de types différents ↔ **surcharge** d'une fonction
  - Choix de la fonction à appeler → dépend de la **signature** de la fonction
  - La fonction surchargée appelée est celle dont la signature est la plus proche des paramètres passées lors de l'appel

Surcharge de fonction
(Uniquement en C++)

- Exemples :
  - Prototypes :

```
double pow ( double base, double exponent );   1
```

Signature

```
float pow ( float base, float exponent );      2
double pow ( double base, int exponent );      3
```

  - Appels :

```
a = pow ( 25.0, 2.0 );    1
b = pow ( 5.1f, 4.0f );   2
c = pow ( 10.2, 2 );      3
d = pow ( 15, 2.5 );      1
```

Valeurs par défaut des arguments
(Uniquement en C++)

- Nouveauté :
  - En C++, il est possible de déclarer, dans les prototypes des fonctions, des valeurs par défaut pour certains paramètres

- Règles :
  - Les paramètres par défaut doivent être les derniers de la liste (i.e. ceux les plus à droite)
  - Les paramètres par défaut sont uniquement déclarés dans le prototype (et non lors de la définition de la fonction)

Valeurs par défaut des arguments
(Uniquement en C++)

- Exemples :
  - Prototypes :

```
double pow ( double base, double exponent = 2.0 );

float volume ( float L, float l = 1.0f, float H = 1.0f );

double rac ( double base = 1.0, int exp );
```

Règle 1 non respectée !

  - Appels :

```
i = pow ( 5.4, 2.5 );

j = volume ( 5.1f, 4.0f, 3.0f );

k = volume ( 1.2, 2.3);

l = pow ( 1.5 );
```